DAY 15 | POST-READ

Testing and Data Engineering: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Testing pyramid	Many unit tests, some integration tests, few end-to-end tests.
PyTest basics	test_*.py files, test_* functions, plain assert, pytest -q.
parametrize and fixtures	One test, many cases; reusable setup injected by name.
API tests	TestClient + fixtures; check status codes and JSON, including errors.
Tests with Codex (NEW v4)	Give the rules, review every expectation, run, then fix code or test.
React tests	Vitest + React Testing Library: render, find by role or text, click, expect.
Data lifecycle	Generate, ingest, store, process, serve.
ETL vs ELT	Transform before loading vs load raw then transform in the warehouse.
Data quality	Complete, unique, valid, fresh. Pipelines must be idempotent.

Part B: Going beyond today

1. Test coverage

Coverage shows which lines your tests never run. High coverage is a guide, not a goal; a few meaningful tests beat many shallow ones. AI-drafted tests can raise coverage without checking anything useful.

```
pip install pytest-cov
pytest --cov=. --cov-report=term-missing
```

2. Test-driven development (TDD)

Write a failing test, write just enough code to pass it, then tidy the code. Red, green, refactor. It pairs well with Codex: you write or approve the test, Codex writes code until it passes.

3. Mocking external calls

Replace slow or paid calls (such as an LLM API) in tests:

```
def test_summary(monkeypatch):
    monkeypatch.setattr("app.llm", lambda p: "short summary")
    assert app.summarise("long text") == "short summary"
```

4. Data quality tools

Tools such as Great Expectations or dbt tests formalise quality rules and run them on every load.

5. Lakes, warehouses and lakehouses

A lake stores raw files cheaply. A warehouse stores clean tables for fast SQL. A lakehouse combines both, which is what Spark is often used for; you meet Spark tomorrow.

Part C: Check yourself

- Why should most tests be unit tests?
- What does a fixture do?
- Why should you give Codex the rules, not only the code, when asking for tests?
- Give one advantage of ELT over ETL.
- What does idempotent mean for a pipeline?

Part D: Practice ideas for tonight

- Reach 10 PyTest tests for your SmartDesk ticket API, including every error case.
- Ask Codex to draft tests for one function, then write a review note on each test.
- Add a data quality test that fails if any ticket has a negative handling time.

Coming up on Day 16: batch data pipelines, Apache Spark and PySpark.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today